

Агентная генерация Use Case и activity-диаграмм по требованиям

с проверяемой трассировкой и ограниченным циклом исправления

Научно-исследовательская работа

Инженерный release candidate и методика экспериментальной оценки. Hidden-оценка и экспертная валидация вынесены в отдельный научный gate.

Исполнитель

Боцман Артём

Направление

Агентные системы и генерация аналитических артефактов

Основной фреймворк

LangGraph

Дата версии

11 сентября 2026 года

Санкт-Петербург, 2026

Аннотация

Разработан воспроизводимый LangGraph pipeline, который принимает структурированный SpecificationReq, формирует Use Cases, user stories, system stories и activity-диаграммы, а затем проверяет структуру и двустороннюю трассировку. В системе работают два агента: Use Case Agent и Activity Diagram Agent. Корневой граф оркестрирует их, материализует TraceManifest и запускает evaluator. LLM-критика и ограниченное исправление дополняют детерминированные валидаторы, но не заменяют их.

Технический benchmark candidate содержит 30 авторских синтетических кейсов: 20 development и 10 hidden. Валидация benchmark проходит для 30/30 кейсов, mutation-suite обнаруживает 15/15 заявленных классов дефектов. Rule-based baseline B0 прогнан на всех 20 DEV-кейсах по три раза. Получено 60/60 технических завершений, stability 1.0 и semantic composite 0.408 с case-level bootstrap 95% CI [0.356; 0.463].

Первый повторный live pilot на DEV-001 сравнил B0, B1 one-shot и FULL. Затем на трёх одинаковых DEV-кейсах разной сложности сравнили пять конфигураций. После уточнения контракта критика FULL завершил 3/3 запуска, получил semantic composite 0.714 и trace F1 0.933. B1 one-shot и вариант без повторного исправления получили 0/3 сквозных завершений. Пилот показывает необходимость контролируемого исправления, но не доказывает преимущество критика: вариант без него также завершил 3/3 запуска.

Статус результатов

Инженерная часть воспроизводима локально. Полный платный DEV, два независимых экспертных review и однократный hidden-run ещё не выполнены и не подменяются синтетическими выводами.

Ключевые термины

Термин	Пояснение
Agent (агент)	Компонент с ролью, входным состоянием, LLM-инструментами и правилами переходов.
Graph (граф)	Исполняемая схема вершин и рёбер, задающая порядок обработки и ветвления.
Baseline (базовая линия)	Упрощённый метод сравнения, относительно которого измеряется вклад решения.
Traceability (трассировка)	Машинные связи от требования к УС, шагам сценария и элементам диаграммы.
Deterministic validator	Не-LLM проверка однозначных структурных правил.
Bounded repair	Исправление с заранее заданным максимальным числом повторов.

1. Постановка задачи

1.1. Контекст и границы

Текущая работа ограничена генерацией аналитико-архитектурных артефактов до написания кода. Исторический проект полного жизненного цикла разработки ПО служит контекстом, но не входит в текущий scope. Это устраняет смешение с агентами-кодерами, тестировщиками и deploy-агентами.

1.2. Цель

Цель работы состоит в разработке и исследовании агентного конвейера, который преобразует подготовленные функциональные и нефункциональные требования в структурированные Use Cases, stories и activity-диаграммы с машинно проверяемой трассировкой.

1.3. Актуальность

Статический анализ эффективно извлекает структуру из существующего кода, но не решает задачу интерпретации бизнес-требований до реализации. LLM способен выделять акторов, цели и сценарии, однако его свободный вывод не гарантирует ссылочную целостность. Поэтому исследуется гибрид: LLM выполняет смысловую работу, а типизированные контракты и детерминированные проверки создают контролируемый контур качества.

1.4. Исследовательские вопросы

- RQ1. Улучшает ли многостадийный pipeline качество по сравнению с one-shot при одинаковом входе, модели и выходном контракте?
- RQ2. Какой вклад дают semantic critic и bounded repair?
- RQ3. Обеспечивает ли typed trace model полноту и ссылочную целостность?
- RQ4. Как меняются качество, стабильность, задержка и расход токенов с ростом сложности требований?

1.5. Критерии завершения

Аспект	Критерий
Pipeline	Не менее 90% DEV завершаются E2E без ручного исправления.
Traceability	100% activity elements имеют источник или явный unsupported status.
UC provenance	Не менее 95% обязательных UC elements имеют FR source или классифицированное происхождение.
Validators	100% подготовленных мутаций поддерживаемых классов обнаружены.
Reproducibility	Повторный расчёт по сохранённым данным даёт идентичные метрики.

2. Входные и выходные контракты

2.1. SpecificationReq

Поле	Тип	Назначение
project_task	str	Исходный запрос пользователя.
project_name	str	Название проектируемого приложения.
project_goal	str	Бизнес-цель.
project_description	str	Контекст и границы продукта.
functional_requirements	list[str]	Функциональные требования, далее атомизируемые в FR.
non_functional_requirements	list[str]	Нефункциональные ограничения и свойства качества.

2.2. Результат одного запуска

- типизированный набор Use Cases и акторов;
- человекочитаемые UC, user stories и system stories;
- типизированные activity-модели и Mermaid source;
- TraceManifest с прямыми и обратными ссылками;
- validation reports и evaluation report;
- run manifest, token/cost telemetry и SQLite checkpoints для FULL.

Почему Mermaid

Mermaid выбран как воспроизводимый текстовый формат визуализации. Внутренняя typed activity model хранится отдельно, поэтому renderer можно заменить. Строгий профиль UML остаётся решением, которое должен утвердить руководитель до scientific freeze.

3. Архитектура решения



Рисунок 1. Полный pipeline и два исполняемых агента.

3.1. Агенты

Компонент	Ответственность	Основной выход
Use Case Agent	Преобразует требования в акторов, UC, шаги сценария и stories.	UseCaseSet, user/system stories
Activity Diagram Agent	Строит поведенческий граф по каждому принятому UC.	ActivityModel, Mermaid
Root orchestrator	Управляет подграфами, объединяет результаты, строит trace и запускает evaluator.	Result bundle

Количество агентов

В проекте два агента. Generator, critic и repair являются внутренними ролями и вершинами подграфов, а не тремя дополнительными автономными агентами.

3.2. Вершины и рёбра

Граф	Вершин	Рёбер	Назначение
Root graph	6	7	Оркестрация UC, activities, trace, evaluation и export.
UC subgraph	9	13	Generation, schema, validation, critic, decision, repair/failure.
Activity subgraph	10	14	Generation, validation, critic, bounded repair, Mermaid.
Итого	25	34	Рабочая статическая структура release candidate.



Рисунок 2. Внутренний ограниченный цикл проверки и исправления.

3.3. Логика ребёр

Обычные рёбра передают типизированное состояние следующей вершине. Условные рёбра читают validation status, critic verdict и число попыток. Результат направляется в accept, repair или controlled failure. Лимит max_repair_attempts исключает бесконечный цикл.

4. Трассировка и детерминированная валидация

Основная цепочка происхождения имеет вид: FR → requirement atom → UC → scenario step → activity node/edge. LLM выбирает типизированные source IDs, а Python детерминированно создаёт единый TraceManifest. Прямые ссылки отвечают на вопрос «что было порождено этим требованием», обратные — «из какого источника появился этот элемент».

4.1. Правила

Правило	Проверка	Зачем
Уникальные ID	Нет повторяющихся FR, UC, step, node, edge ID.	Предотвращает неоднозначные ссылки.
Типы концов связи	Каждая связь соединяет допустимые типы сущностей.	Исключает логически невозможные trace links.
Ссылочная целостность	Source и target реально существуют.	Выявляет dangling references.
Достижимость	Все activity nodes достижимы от start.	Исключает изолированные фрагменты.
Branch guards	У ветвлений есть условия перехода.	Делает сценарий интерпретируемым.
Provenance	Unsupported/assumption помечены и объяснены.	Не скрывает неподтверждённые элементы.

4.2. Почему валидатор не заменяется критиком

LLM-критик полезен для смысловой полноты и согласованности, но вероятностен. Он не даёт строгой гарантии уникальности ID, достижения 100% coverage или существования каждой ссылки.

Детерминированный валидатор повторяет одну и ту же проверку с одинаковым результатом, поэтому является обязательным gate до принятия артефакта.

Validator mutation suite v1
15 predefined structural and trace error classes; no LLM calls

15/15 detected (100%)

Covered: IDs, FR atoms, forward/reverse links, endpoint types, dangling links, control flow, guards, partitions and provenance.

Scope limit: this result does not prove semantic correctness or detection of unenumerated error classes.

Рисунок 3. Mutation-suite: 15 из 15 заявленных дефектов обнаружены.

5. Benchmark и методика оценки



Рисунок 4. Состав benchmark candidate v1.0.

Характеристика	Значение
Всего кейсов	30
Разбиение	20 DEV / 10 hidden
Сложность	8 simple / 13 medium / 9 hard
Язык	13 RU / 17 EN
Gold	Акторы, UC, milestones, branches, FR-to-UC links
Эквивалентность	Canonical slots, aliases, optional slots, graph-equivalence rules

5.1. Метрики

Группа	Метрика	Интерпретация
Формат	Schema / structural validity	Прошёл ли результат типизированные контракты.
Trace	Integrity, forward/reverse coverage	Полнота и корректность происхождения.
Семантика	Actor/UC/milestone/branch F1	Совпадение смысловых gold slots с эквивалентностями.
Риск	Hallucination proxy	Доля неподтверждённого содержания.
Система	E2E success	Прошёл ли весь обязательный контур.
Цена	Latency, calls, tokens, cost, repairs	Ресурсная стоимость качества.
Устойчивость	Multi-run stability	Сходство результатов повторных запусков.

Ограничение composite

Semantic composite используется как dashboard metric. Он не заменяет отдельные проверки, E2E gate и экспертную оценку. Пилот показал, что B1 может иметь более высокий composite, но формально не пройти трассировку.

5.2. Экспертная шкала

Балл	Смысл
1	Критически неверно: результат нельзя использовать без полной переработки.
2	Существенные ошибки: пропущены ключевые акторы, цели или ветви.
3	Частично корректно: основа верна, но нужны заметные исправления.
4	Почти корректно: только локальные правки без изменения структуры.
5	Корректно и пригодно к использованию в заявленной постановке.

6. Сравнимые методы

Метод	Вход	LLM	Critic / repair	Роль в исследовании
B0_RULE	SpecificationReq	Нет	Нет	Детерминированный нижний ориентир.
B1_ONESHOT	SpecificationReq	Да, один вызов	Нет	Прямой baseline той же модели.
FULL	SpecificationReq	Да	Да	Основной двухагентный pipeline.
FULL_NO_CRITIC	SpecificationReq	Да	Только validator/repair	Вариант без отдельного компонента semantic critic.
FULL_NO_REPAIR	SpecificationReq	Да	Critic, 0 repair	Вариант без отдельного компонента bounded repair.
Pyreverse	Python-код	Нет	Нет	Adjacent baseline для code-to-UML.
GitDiagram / DeepWiki	Репозиторий	Да	Сервисный контур	Смежные repository-to-diagram системы.

7. Экспериментальные результаты

7.1. Repeated live pilot DEV-001

Метод	E2E	Semantic	Activity trace	Tokens/run	Repairs/run	n
B0	2/2	0.633	1.000	0	0	2
B1	0/2	0.724	0.881	8 969	0	2
FULL	2/2	0.670	1.000	33 445	2	2

B1 не прошёл оба запуска из-за непротраассированных activity edges. Error analysis выявил 8 случаев `activity_edge_untraced` и 2 случая `activity_trace_coverage_below_threshold`. FULL обнаружил четыре промежуточных `activity_edge_untraced` и исправил их; в финальном результате неразрешённых blocking issues не осталось.

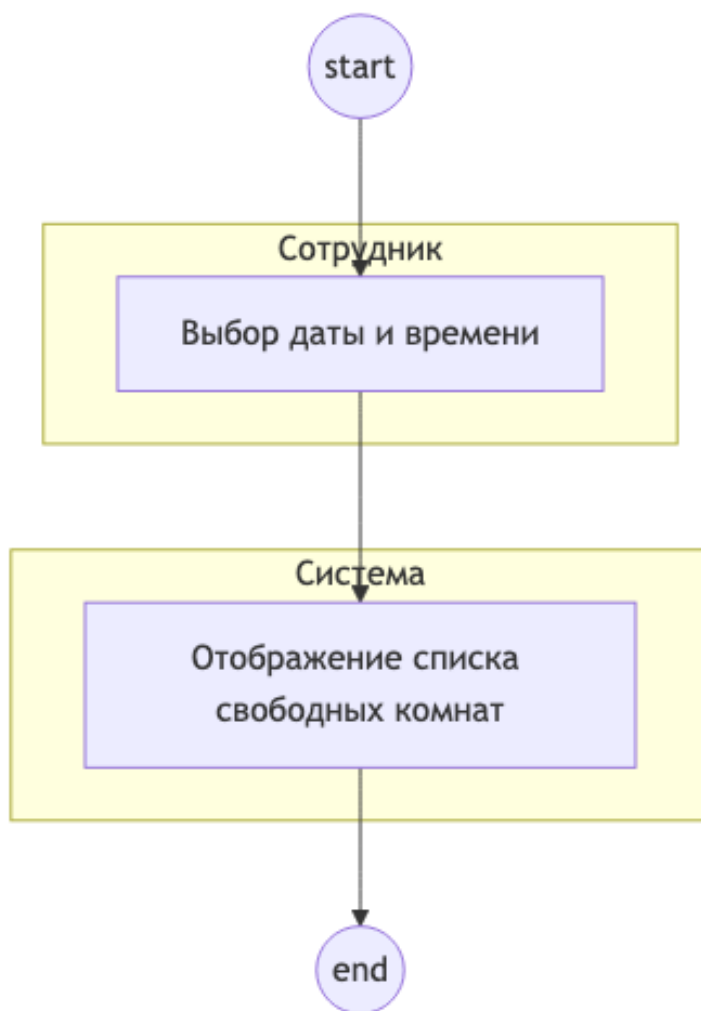


Рисунок 5. Пример сгенерированной activity-диаграммы DEV-001, UC-001.

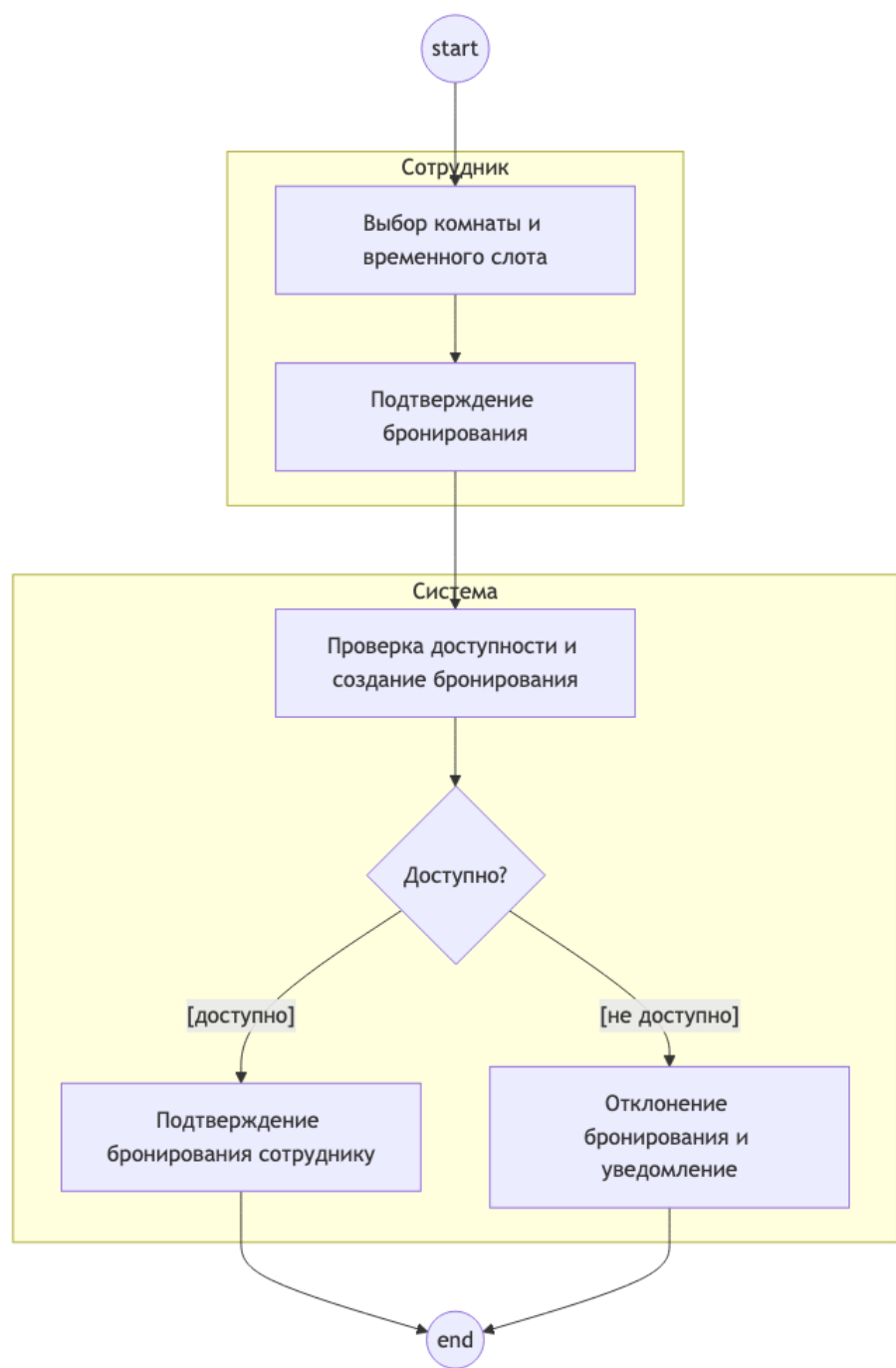


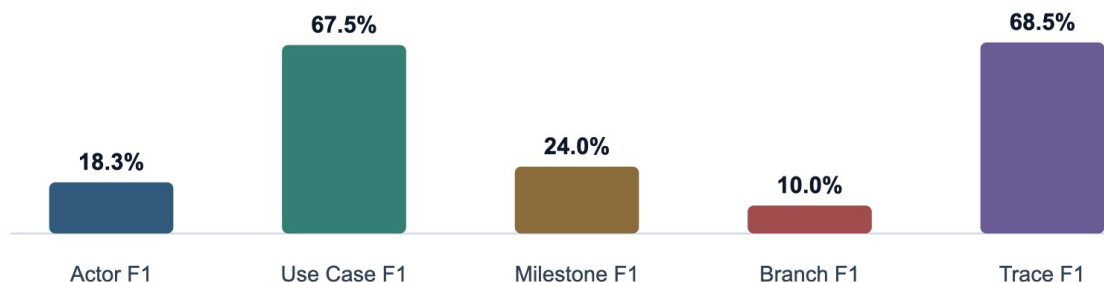
Рисунок 6. Пример сгенерированной activity-диаграммы DEV-001, UC-002.

7.2. Полный В0 на DEV20

Группа	Кейсов	Запусков	E2E	Semantic	95% CI	Hallucination
Все DEV	20	60	1.000	0.408	[0.356; 0.463]	0.493
Simple	6	18	1.000	0.543	[0.481; 0.601]	0.396
Medium	9	27	1.000	0.359	[0.293; 0.433]	0.522
Hard	5	15	1.000	0.335	[0.296; 0.372]	0.556

B0 deterministic baseline — full DEV20

20 cases × 3 identical repeats; 60/60 completed; 0 LLM tokens; hidden tests untouched



Quality by case complexity

simple (n=6): semantic 54.3%; hallucination proxy 39.6%

medium (n=9): semantic 35.9%; hallucination proxy 52.2%

hard (n=5): semantic 33.5%; hallucination proxy 55.6%

Conclusion: deterministic construction is reproducible and structurally valid, but not semantically sufficient.

Рисунок 7. Качество B0 по сложности требований.

Все 60 запусков завершились технически, stability равна 1.0. Semantic composite ниже 0.5 получен у 15 из 20 уникальных кейсов; из-за трёх одинаковых повторов это соответствует 45 из 60 запусков. Hallucination proxy выше 0.4 получен в 48 из 60 запусков. Следовательно, B0 подтверждает работоспособность инфраструктуры и воспроизводимость, но не решает смысловую задачу.

7.3. Сравнение пяти конфигураций на трёх DEV-кейсах

В пилот вошли B1-DEV-002, B1-DEV-010 и B1-DEV-020: простой русский, средний русский и сложный английский кейсы. Для каждой конфигурации выполнено по одному запуску на кейс. Все LLM-конфигурации использовали deepseek-flash, temperature=0, один endpoint и одинаковые входы; hidden не открывался.

Метод	E2E	Semantic	Branch F1	Trace F1	Tokens/run	USD/run
B0	3/3	0.524	0.000	0.857	0	0.0000
B1	0/3	0.487	0.556	0.619	10 358	0.0046
FULL	3/3	0.714	0.690	0.933	52 947	0.0294
Без критика	3/3	0.699	0.500	0.952	37 011	0.0238
Без	0/3	0.730	0.786	0.952	9 380	0.0065

исправления					
-------------	--	--	--	--	--

Сравнение методов на трёх DEV-кейсах

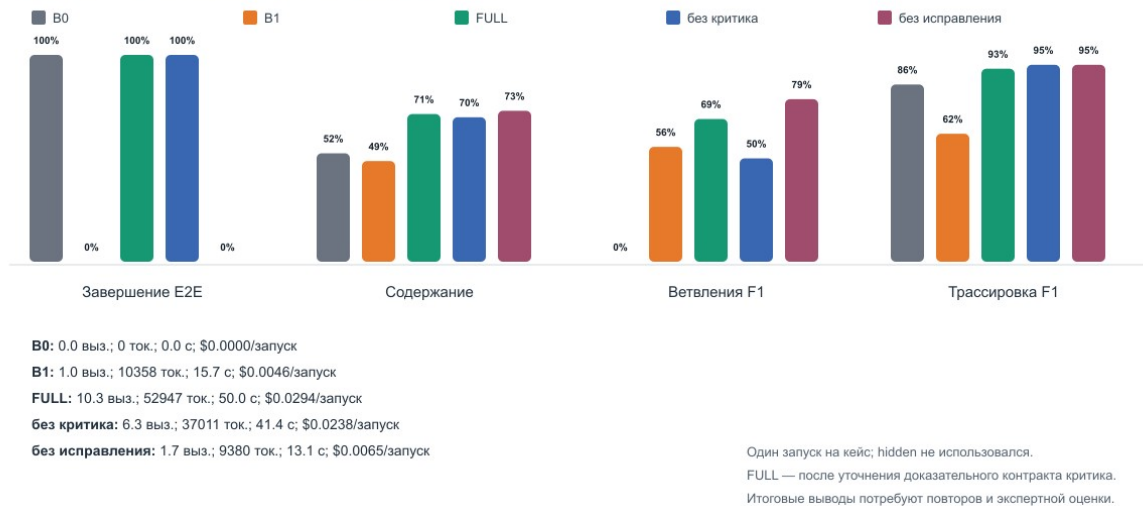


Рисунок 8. Прямое сравнение пяти конфигураций на одинаковых DEV-входах.

B0 проходит структурный барьер, но не восстанавливает ветвления. Вариант без исправления получил высокий автоматический содержательный балл, но не сформировал принимаемый полный пакет. Поэтому semantic composite нельзя использовать без E2E gate. Прямой one-shot оказался примерно в 6.4 раза дешевле FULL, но не дал ни одного принимаемого результата на выбранных кейсах.

7.4. Исправление контракта критика

Первоначальный FULL завершил только 1/3 запусков: критик мог вводить новый критерий после каждого исправления. Теперь блокирующее замечание обязано ссылаться на конкретное требование и элемент модели, а замечания к стилю не блокируют результат. Завершение выросло до 3/3, но средняя стоимость - с 0.0239 до 0.0294 USD. Это подтверждает устранение инженерной ошибки, но не является статистическим доказательством полезности критика.

Контракт критика: до и после уточнения



Рисунок 9. Работа FULL до и после уточнения доказательного контракта критика.

8. Воспроизводимость, логи и безопасность

8.1. Сохраняемые доказательства

- config.json с режимом и безопасной конфигурацией;
- generated specification, UC, stories, activity JSON и Mermaid;
- TraceManifest, validation/evaluation reports;
- calls telemetry, latency, token usage, repair count;
- SQLite checkpoints для возобновления FULL;
- results.json/results.csv, SHA-256 и offline reproducibility report.

8.2. Сетевые ошибки и budget guards

OpenAI-compatible adapter повторяет только временные ошибки 408, 409, 425, 429 и 5xx, учитывает Retry-After и ограничивает задержку. Ошибки 400, 401 и 403 не повторяются. Перед live experiment задаются максимальные calls, tokens и estimated cost.

8.3. Raw evidence

Публичный result bundle хранит хэши prompt/response и sanitized telemetry. Точные request/response можно сохранить только явно через --capture-raw-private-dir. Приватный каталог получает права 700, файлы — 600, API key не записывается, а путь не попадает в публичный config. Каталог .private_experiment_evidence исключён из Git.

8.4. Повторный расчёт

Offline verifier заново вычисляет summary, stability и SHA-256 по сохранённым bundles. Для B0 DEV20 подтверждены 60 ожидаемых и 60 полных bundles. Для repeated DEV-001 подтверждены 6 из 6. Исторически сохранённые значения совпали; новые производные поля отчётливо отмечены как дополнительные, а не как расхождение.

9. Готовность по плану НИР

Этап	Формальная готовность	Готово	Внешний или платный gate
Планирование	95%	План, RQ, риски, контрольные точки.	Подтвердить сроки и план.
Анализ и проектирование	95%	Контракты, 2 агента, 25 вершин, 34 ребра.	Утвердить UML/trace policy.
Benchmark и методика	90%	30 кейсов, gold, split, freeze, evaluator.	2 эксперта и разрешение публикации.
Pipeline	85%	FULL, B0/B1, component variants, SQLite, retry, CLI.	Полный paid FULL DEV20 и $\geq 90\%$.
Трассировка	90%	TraceManifest и mutation 15/15.	Согласование правил и full DEV.
Интеграция	80%	RC, B0 DEV20×3, пять вариантов на DEV-3, verifier.	Повторный FULL DEV20.
Эксперименты	55%	B0 full и пять вариантов на трёх DEV-кейсах.	Повторы DEV20, 2 эксперта, hidden.
Финализация	95%	README, CI, report, deck, speech, demo.	Репетиция и публикационное решение.

Что означает «доделать полностью»

Код, воспроизводимость и материалы защиты можно завершить автономно. Научный DoD нельзя объявить 100%, пока реальные эксперты не поставили оценки, руководитель не утвердил правила, а платный repeated DEV и запечатанный hidden-run не были выполнены по согласованному протоколу.

10. Ограничения валидности

- Benchmark создан автором и имеет technical author freeze, но ещё не прошёл два независимых review.
- Прямое сравнение пяти конфигураций охватывает три DEV-кейса и один запуск на кейс.
- Semantic matcher и его порог не откалиброваны по экспертам.
- Alias модели провайдера не гарантирует immutable model revision.
- Порог 90% требует полного FULL DEV20, обобщаемость — однократного hidden-run после freeze.
- НФТ сохраняются и проверяются на уровне контракта/происхождения, но не все НФТ визуализируются activity-диаграммой.

11. План завершения эксперимента

Шаг	Действие	Условие перехода
1	Утвердить MODEL_ID/API_BASE, UML profile, trace policy, benchmark publication.	Письменное решение руководителя.
2	Повторить FULL и варианты без отдельных компонентов на DEV20.	Нет блокирующих pipeline/evaluator ошибок.
3	Рассчитать доверительные интервалы, устойчивость и анализ классов ошибок.	Метрики пересчитываются из сохранённых bundles.
4	Два независимых expert review, agreement и scientific freeze.	Зафиксированы commit/tag/hash.
5	Однократный hidden-run без настройки по результату.	Scientific freeze завершён.
6	Финальная интерпретация, репетиция и публикация разрешённой части.	Все числа восстанавливаются из bundles.

12. Вывод

Разработан полный исполняемый контур от SpecificationReq до трассируемых activity-диаграмм и отчёта качества. Детерминированные проверки обнаруживают структурные дефекты, а ограниченный agentic repair способен исправить непротрассированные элементы в live pilot. В0 доказывает воспроизводимость методики, но показывает недостаточность простых правил для смысловой задачи.

Инженерный release candidate готов к демонстрации. Пилот показывает вклад ограниченного исправления и цену многостадийного контура, но не позволяет объявить статистическое превосходство FULL или полезность критика. Научное завершение требует согласования моделей и правил, двух независимых экспертов, повторного FULL DEV20 и единственного hidden-run после научной фиксации.

Литература и источники

1. Hugging Face Agents Course. Unit 0 и LangGraph unit.
<https://huggingface.co/learn/agents-course/unit0/introduction>
2. LangGraph documentation. Overview, persistence and graph design.
<https://docs.langchain.com/oss/python/langgraph/overview>
3. ISO/IEC/IEEE 29148:2024. Requirements engineering. <https://www.iso.org/standard/72089.html>
4. OMG UML 2.5.1 specification. <https://www.omg.org/spec/UML/2.5.1/PDF>
5. Automated Generation of SysML Activity Diagrams from Industrial Requirements Using LLMs. RE 2026.
<https://conf.researchr.org/details/RE-2026/RE-2026-industrial-innovation-papers/4/>
6. R2ABench: requirements-to-architecture evaluation. 2026. <https://arxiv.org/abs/2604.06683>
7. TraceLLM benchmark. 2026. <https://arxiv.org/abs/2602.01253>
8. Systematic mapping of LLMs in model-driven engineering. 2026.
<https://link.springer.com/article/10.1007/s10664-026-10921-4>
9. Text2UML evaluation artifacts and scenarios. 2025. <https://github.com/IlKaiser/text2uml>
10. Position bias in LLM-as-a-judge. 2025. <https://aclanthology.org/2025.ijcnlp-long.18/>

Приложение А. Проверяемые артефакты

Артефакт	Расположение в репозитории
Benchmark	benchmark/v1_0_synthetic/cases.json
Freeze record	benchmark/v1_0_synthetic/freeze_record.json
Методика	docs/benchmark/BENCHMARK_AND_METRICS_V0_1.md
Экспертная рубрика	benchmark/expert_review/RUBRIC.md
System Design	docs/architecture/SYSTEM_DESIGN_V0_1.md
Traceability Model	docs/architecture/TRACEABILITY_MODEL.md
B0 DEV20	artifacts/benchmark_runs/b0-dev20-r3-2026-09-09
Repeated live pilot	artifacts/benchmark_runs/repeated-b0-b1-full-dev001-2026-09-09
Five-method DEV pilot	artifacts/technology_project_evidence_2026-09-11
Full critic v2 run	artifacts/benchmark_runs/full-critic-v2-dev3-deepseek-flash-2026-09-11
Comparison report	docs/research/TECHNOLOGY_PROJECT_COMPARISON_2026_09_11.md
Reproducibility verifier	scripts/verify_saved_experiment.py
Error analysis	scripts/analyze_saved_experiment.py